

C++ REST API Framework

Constantin Diana-Gabriela
Academia Tehnică Militară „Ferdinand I”
București, România

Stelea Emilia
Academia Tehnică Militară „Ferdinand I”
București, România

I. INTRODUCERE

În prezent, majoritatea aplicațiilor moderne comunică între ele prin intermediul unor servicii web, care permit schimbul de date într-un mod standardizat și ușor de extins. Unul dintre cele mai utilizate stiluri arhitecturale pentru realizarea acestor servicii este REST (Representational State Transfer), care definește un set de principii pentru construirea aplicațiilor de tip client–server. În cadrul acestui stil, interacțiunea dintre client și server se realizează prin cereri HTTP, utilizând metode precum GET, POST, PUT și DELETE, iar datele sunt transmise, de regulă, într-un format ușor de prelucrat, cum este JSON.

Un framework poate fi definit ca o structură software care oferă un set de componente, reguli și mecanisme de bază menite să sprijine dezvoltarea aplicațiilor, reducând efortul necesar implementării de la zero a funcționalităților comune. În cazul unui framework pentru REST API, acesta oferă suport pentru gestionarea cererilor HTTP, organizarea logicii aplicației pe straturi și facilitarea comunicării cu un strat de date persistent, responsabil de stocarea și gestionarea informațiilor într-o bază de date.

Proiectul „C++ REST API Framework” urmărește realizarea unui framework minimal pentru dezvoltarea de aplicații de tip REST API, implementat în limbajul C++. Scopul proiectului nu este crearea unui produs complet comparabil cu framework-uri consacrate, ci înțelegerea modului în care astfel de soluții pot fi construite utilizând mecanisme specifice sistemelor de operare. Proiectul integrează noțiuni precum procese, thread-uri, mecanisme de sincronizare și comunicare inter-proces, într-un context realist, specific aplicațiilor de tip server. Astfel, aplicația este capabilă să gestioneze mai multe conexiuni simultane și să proceseze cereri concurente într-un mod controlat.

Aplicația acceptă cereri HTTP de tip GET, POST, PUT și DELETE și oferă răspunsuri în format JSON, datele fiind stocate persistent într-o bază de date SQLite. Structura internă a aplicației este organizată pe straturi logice distincte, corespunzătoare stratului de date (Data Layer), stratului de servicii (Service Layer) și stratului de control (Controller Layer).

Obiectivele principale ale proiectului sunt:

- înțelegerea principiilor de bază ale arhitecturii REST și ale dezvoltării de API-uri;
- proiectarea unui framework minimal pentru REST API în limbajul C++;
- implementarea unui server capabil să gestioneze cereri HTTP concurente;

- organizarea aplicației pe straturi logice, conform unui pattern arhitectural consacrat;
- utilizarea unui strat de date persistent pentru gestionarea informațiilor.

II. RELATED WORK

Dezvoltarea aplicațiilor de tip REST API reprezintă un domeniu intens studiat atât în mediul academic, cât și în industrie, datorită rolului esențial pe care aceste servicii îl joacă în arhitecturile moderne client–server. Stilul arhitectural REST a fost introdus și formalizat de Roy T. Fielding, care a definit principiile fundamentale ce stau la baza interacțiunii dintre client și server prin intermediul protocolului HTTP. Aceste principii au stat la baza majorității serviciilor web moderne.

În practică, dezvoltarea aplicațiilor REST este realizată frecvent cu ajutorul unor framework-uri de nivel înalt, precum Spring Boot (Java), Django REST Framework (Python) sau Express.js (Node.js). Aceste soluții oferă mecanisme avansate pentru rutarea cererilor, validarea datelor, securitate și integrarea cu baze de date, reducând semnificativ efortul de dezvoltare.

În literatura de specialitate, modele precum thread-per-request, thread pool sau master–worker sunt utilizate pentru a gestiona eficient un număr mare de cereri simultane. Aceste modele sunt analizate în contextul performanței, al consumului de resurse și al scalabilității, fiind frecvent implementate folosind mecanisme POSIX în sistemele UNIX-like. Mecanisme precum memoria partajată și semafoarele POSIX sunt soluții eficiente pentru schimbul de date între procese, cu condiția unei sincronizări corecte.

În ceea ce privește persistența datelor, SQLite este apreciată pentru simplitatea integrării, lipsa necesității unui server dedicat și suportul bun pentru limbajul C/C++, fiind menționată frecvent în lucrări care abordează aplicații client–server minimaliste.

III. ARHITECTURA ȘI DESIGNUL SISTEMULUI

Aplicația dezvoltată în cadrul proiectului „C++ REST API Framework” este concepută ca un server de tip client–server, capabil să gestioneze cereri HTTP concurente și să ofere răspunsuri în format JSON. Din punct de vedere arhitectural, soluția adoptată urmărește separarea clară a responsabilităților și organizarea logicii aplicației pe straturi distincte, facilitând atât înțelegerea funcționării sistemului, cât și extinderea acestuia.

Pentru atingerea acestor obiective, aplicația este structurată conform pattern-ului arhitectural MVCS

(Model–View–Controller–Service). Acest pattern reprezintă o extensie a modelului MVC clasic, prin introducerea unui strat suplimentar de servicii, responsabil de implementarea logicii. Alegerea acestui pattern este justificată de necesitatea separării clare între procesarea cererilor, logica aplicației și accesul la date.

În cadrul aplicației, fiecare componentă a pattern-ului MVCS îndeplinește un rol bine definit:

- Controller Layer (controller) – este responsabil de preluarea cererilor HTTP primite de la clienți, analizarea metodei și a rutei solicitate, precum și de direcționarea cererilor către stratul de servicii corespunzător. Controller-ul reprezintă punctul de intrare în aplicație pentru cererile externe.
- Service Layer (service) – implementează logica aplicației. Acest strat conține operațiile care definesc comportamentul aplicației, fiind independent de modul de primire a cererilor sau de detaliile de stocare a datelor. Service Layer acționează ca un intermediar între Controller și Model.
- Model / Data Layer (repository, database) – este responsabil de accesul la datele persistente. În cadrul proiectului, acest strat gestionează interacțiunea cu baza de date SQLite, execută interogări SQL și mapează rezultatele în structuri C++.
- View / Response Layer (`http_response`) – se ocupă de construirea răspunsurilor HTTP trimise către client. Acest strat este reprezentat de componenta care generează răspunsurile JSON și codurile de stare HTTP corespunzătoare.

Această organizare permite izolarea modificărilor la nivelul unui strat, fără a afecta celelalte componente ale aplicației, contribuind astfel la o arhitectură mai flexibilă și mai ușor de întreținut.

Pe lângă structura logică bazată pe pattern-ul MVCS, arhitectura aplicației include și un model de execuție concurent, necesar pentru gestionarea eficientă a cererilor HTTP multiple. Din punct de vedere structural, aplicația este alcătuită din două tipuri principale de componente de execuție:

- Serverul principal (Master) – responsabil de acceptarea conexiunilor de la clienți și de primirea cererilor HTTP.
- Procesele worker – responsabile de procesarea efectivă a cererilor primite și de generarea răspunsurilor.

Serverul ascultă conexiuni pe un socket TCP și poate gestiona simultan mai mulți clienți, folosind un mecanism de tip thread pool. Pentru a permite procesarea paralelă a cererilor, serverul delegă task-urile către procese worker separate, utilizând un model de tip master–worker.

Comunicarea dintre server și workeri este realizată prin mecanisme de comunicare inter-proces (IPC), bazate pe memorie partajată și semafoare POSIX. Acest mecanism permite transmiterea cererilor și a răspunsurilor într-un mod sincronizat.

Fluxul de procesare a unei cereri HTTP poate fi descris prin următorii pași:

- Clientul trimite o cerere HTTP către serverul principal.

Client

|

HTTP

|

Server (Master + Thread Pool)

|

IPC

|

Worker (Thread Pool)

|

Controller -> Service -> Repository -> SQLite

Figura 1. Modelul arhitectural client-server și master-worker

- Serverul acceptă conexiunea și preia cererea folosind un thread din thread pool.
- Cererea este plasată într-o structură de date partajată, accesibilă proceselor worker.
- Un proces worker preia cererea și o procesează în mod concurent, utilizând propriul thread pool.
- Controller-ul analizează cererea și apelează stratul de servicii corespunzător.
- Service Layer interacționează cu Data Layer pentru accesul la baza de date.
- Răspunsul este generat și transmis înapoi către serverul principal.
- Serverul trimite răspunsul HTTP către client.

Acest flux permite procesarea eficientă a mai multor cereri simultane, distribuind sarcina de lucru între mai multe thread-uri și procese.

IV. IMPLEMENTARE

Aplicația a fost implementată în limbajul C++, utilizând facilități specifice sistemelor de operare POSIX pentru gestionarea proceselor, thread-urilor și mecanismelor de sincronizare.

A. Limbaje, tehnologii și biblioteci utilizate

Aplicația este implementată în limbajul C++, ales pentru nivelul ridicat de control asupra resurselor sistemului și pentru suportul nativ oferit în mediile UNIX-like. Utilizarea C++ permite gestionarea directă a memoriei, a thread-urilor și a

mecanismelor de sincronizare, aspecte esențiale pentru un server concurrent.

Pentru implementarea concurenței, proiectul utilizează POSIX Threads (pthreads), care permit crearea și gestionarea eficientă a thread-urilor în cadrul serverului principal și al proceselor worker. Acest mecanism este utilizat sub forma unui thread pool, reducând costul asociat creării și distrugerii frecvente a thread-urilor. Separarea execuției între serverul principal și procesele worker este realizată folosind procese POSIX, prin apeluri de sistem specifice (fork, exec). Comunicarea dintre aceste procese este realizată prin mecanisme de IPC POSIX, bazate pe memorie partajată și semafoare, care asigură sincronizarea accesului la datele comune.

Pentru comunicarea client-server sunt utilizate socket-uri TCP, care oferă un canal de comunicație fiabil pentru transmiterea cererilor HTTP. La nivel de aplicație, serverul implementează un mecanism minimal de procesare a protocolului HTTP.

Persistența datelor este asigurată prin utilizarea bazei de date SQLite, care permite stocarea informațiilor într-un fișier local, fără necesitatea unui server de baze de date dedicat. Accesul la baza de date este realizat printr-un strat dedicat, care izolează detaliile interogărilor SQL de restul aplicației.

B. Structura proiectului

Structura proiectului este organizată la nivel de fișiere sursă, fiecare componentă principală a aplicației fiind implementată într-un fișier separat. Această abordare permite separarea clară a responsabilităților și reflectă arhitectura logică a sistemului.

Fișierele controller și service implementează stratul de control și logica aplicației, fiind responsabile de procesarea cererilor și coordonarea operațiilor de business. Accesul la datele persistente este realizat prin fișierele repository și database, care gestionează interacțiunea cu baza de date SQLite și izolarea detaliilor de stocare.

Componentele legate de execuția concurentă și comunicarea inter-proces sunt implementate în fișiere dedicate, precum server și worker, care gestionează rolurile de master și worker, respectiv thread_pool, shared_queue și semaphore, care asigură mecanismele necesare pentru paralelizare și sincronizare.

Generarea răspunsurilor HTTP este realizată de componenta http_response, iar funcționalitățile auxiliare sunt grupate în fișierul utils. Fișierul Makefile este utilizat pentru automatizarea procesului de compilare, iar baza de date users.db asigură stocarea persistentă a informațiilor utilizate de aplicație.

Această organizare modulară la nivel de fișiere permite o bună lizibilitate a codului și facilitează extinderea sau reorganizarea ulterioară a proiectului.

C. Protocolul de comunicare și formatele de date

Comunicarea dintre client și server se realizează prin intermediul protocolului HTTP, peste o conexiune TCP. Cererile HTTP sunt interpretate de server printr-un mecanism de procesare minimal care permite extragerea metodei, rutei și a conținutului

C++ Rest API Framework/



Figura 2. Structura proiectului

mesajului. Datele transmise în cadrul cererilor și răspunsurilor sunt reprezentate în format JSON. Metodele HTTP suportate de aplicație sunt:

- GET – pentru obținerea datelor;
- POST – pentru crearea de resurse noi;
- PUT – pentru actualizarea resurselor existente;
- DELETE – pentru ștergerea resurselor.

Datele sunt transmise între client și server în format JSON, acest format fiind ales datorită lizibilității și ușurinței de prelucrare. Răspunsurile generate de server includ atât coduri de stare HTTP, cât și conținutul JSON corespunzător operației efectuate.

D. Gestionarea concurenței și a execuției

Pentru a susține procesarea simultană a mai multor cereri, aplicația utilizează un model de execuție concurent, bazat pe combinația dintre thread-uri și procese. Serverul principal folosește un thread pool pentru a gestiona conexiunile multiple ale clienților, fiecare cerere fiind procesată într-un thread separat.

Procesarea efectivă a cererilor este delegată către procese worker, care la rândul lor utilizează un thread pool intern pentru paralelizarea execuției. Comunicarea dintre server și workeri este realizată prin mecanisme IPC POSIX, bazate pe memorie partajată și semafoare, asigurând sincronizarea corectă a accesului la datele partajate.

E. Accesul la date și persistența

Stratul de acces la date este implementat folosind baza de date SQLite, care oferă suport pentru stocarea persistentă a informațiilor. Interacțiunea cu baza de date este realizată printr-un strat de tip Repository, care izolează detaliile de implementare ale interogărilor SQL de restul aplicației. Această separare permite modificarea sau înlocuirea mecanismului de stocare a datelor fără a afecta logica de business sau stratul de control, contribuind la flexibilitatea generală a framework-ului.

V. TESTARE ȘI VALIDARE

A. Testare funcțională și de integrare

Testarea funcțională a fost realizată prin trimiterea de cereri HTTP către server folosind utilitarul curl, rulat dintr-un terminal separat, pe aceeași mașină virtuală pe care rulează aplicația server. Această abordare a permis verificarea corectitudinii procesării cererilor și a interacțiunii dintre componentele aplicației (server, worker, controller, service și stratul de date).

Prin aceste teste au fost validate:

- interpretarea corectă a metodelor HTTP (GET, POST, PUT, DELETE);
- rutarea cererilor către componentele corespunzătoare;
- generarea răspunsurilor în format JSON;
- persistența datelor în baza de date SQLite.

B. Scenarii de testare

Au fost testate următoarele scenarii:

- Adăugarea unui utilizator (POST) Trimiterea unei cereri de tip POST cu date valide duce la inserarea unui nou utilizator în baza de date și la returnarea unui mesaj de confirmare.
- Obținerea listei de utilizatori (GET) Trimiterea unei cereri de tip GET returnează lista utilizatorilor existenți, în format JSON.
- Actualizarea unui utilizator (PUT) Trimiterea unei cereri de tip PUT pentru un utilizator existent duce la modificarea corectă a datelor acestuia în baza de date.
- Ștergerea unui utilizator (DELETE) Trimiterea unei cereri de tip DELETE pentru un utilizator existent duce la eliminarea acestuia din baza de date.

În situația în care se încearcă ștergerea sau actualizarea unui utilizator inexistent, aplicația returnează un mesaj corespunzător, iar baza de date nu este modificată. Acest comportament confirmă tratarea corectă a cazurilor limită și menținerea consistenței datelor.

```
diana@diana-VirtualBox:~/Desktop/PFinal2/PPS02$ curl -X GET http://localhost:8080/users
[{"Ana", "Mihai", "Diana", "Emilia", "Zuzu", "Decembrie", "Andrei", "Ema", "Ianuarie"}]
diana@diana-VirtualBox:~/Desktop/PFinal2/PPS02$ curl -X DELETE http://localhost:8080/users -d "name=
Gabriel"
{"error": "User not found"}
diana@diana-VirtualBox:~/Desktop/PFinal2/PPS02$
```

Figura 3. Testarea folosind comenzi curl pentru operațiile GET și DELETE

C. Concurență

Pentru evaluarea capacității aplicației de a gestiona cereri concurente, au fost trimise mai multe cereri HTTP succesive, din terminale diferite. Serverul a procesat corect cererile, utilizând mecanismele de tip thread pool și modelul master-worker, fără blocări sau comportamente neașteptate.

D. Fiabilitate și consistența datelor

Persistența datelor a fost verificată prin repornirea aplicației server și reexecutarea cererilor de tip GET, observându-se că datele salvate anterior sunt păstrate în baza de date SQLite. De asemenea, în cazul cererilor invalide, baza de date nu este afectată, ceea ce confirmă menținerea consistenței informațiilor.

VI. CONCLUZII ȘI PERSPECTIVE

În cadrul acestui proiect a fost realizat un framework minimal pentru dezvoltarea de aplicații REST API în limbajul C++. Aplicația implementată permite gestionarea cererilor HTTP de tip GET, POST, PUT și DELETE și oferă răspunsuri în format JSON, utilizând o arhitectură modulară și un model de execuție concurent.

Rezultatele obținute demonstrează că soluția propusă îndeplinește cerințele funcționale inițiale ale proiectului. Structurarea aplicației conform pattern-ului MVCS a permis separarea clară a responsabilităților între straturile de control, servicii și acces la date, contribuind la o arhitectură coerentă și ușor de urmărit. Utilizarea mecanismelor POSIX pentru procese, thread-uri și comunicare inter-proces a permis procesarea concurentă

a cererilor și a asigurat un comportament stabil al aplicației în scenarii cu cereri multiple.

Ca direcții viitoare de dezvoltare, framework-ul ar putea fi extins prin: implementarea unui parser HTTP mai complet și mai robust, introducerea unor mecanisme avansate de gestionare a erorilor și a codurilor de stare, realizarea unor teste automate și a unor teste de performanță mai riguroase.

În concluzie, soluția obținută reprezintă un punct de plecare potrivit pentru dezvoltări ulterioare și pentru aprofundarea conceptelor studiate în cadrul disciplinei Proiectarea Sistemelor de Operare.

BIBLIOGRAFIE

- [1] R. T. Fielding, "REST APIs must be hypertext-driven," *IEEE Internet Computing*, vol. 12, no. 5, pp. 85–86, 2008. Available: <https://roy.gbiv.com/untangled/2008/rest-apis-must-be-hypertext-driven>
- [2] IETF, "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content," RFC 7231, 2014. Available: <https://datatracker.ietf.org/doc/html/rfc7231>
- [3] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, 2018.
- [4] The Open Group, "Interprocess Communication (IPC)," IEEE Std 1003.1. Available: https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap04.html
- [5] SQLite Consortium, "SQLite C Interface – An Introduction." Available: <https://www.sqlite.org/cintro.html>